

C others

https://github.com/kkh3363/C_Lang

Contents

01 Rand()

02 Sorting

01 Rand()

C `stdlib rand()` Function

https://github.com/kkh3363/C_Lang

■ Definition and Usage

- The `rand()` function returns a random non-negative integer.
- Numbers generated by this function are pseudo-random, which means that it is possible to predict them with enough information.
- To make the numbers appear more random they should be seeded using the `srand()` function with a different value each time the program runs.
- The `rand()` function is defined in the `<stdlib.h>` header file.

rand() Function

https://github.com/kkh3363/C_Lang

■ Example

- Display 10 random numbers between 1 and 100:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int i, num;
    // Initialize the randomizer using the current timestamp as a seed
    // (The time() function is provided by the <time.h> header file)
    srand(time(NULL));

    // Generate random numbers
    for( i = 0; i < 10; i++) {
        num = rand() % 100 + 1;
        printf("%d ", num);
    }

    return 0;
}
```

Example

https://github.com/kkh3363/C_Lang

■ Rock, Paper, Scissors

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int user, computer;
    char* choices[] = {"", "Rock", "Paper", "Scissors"};
    // initialize rand
    srand((unsigned int)time(NULL));
    printf("=== Rock , Paper, Scissors Game ===\n");
    printf("1. Rock 2. Paper 3. Scissors 4. Exit\n");
    printf("Select # : ");
    scanf("%d", &user);
    // game exit
    if (user == 4) {
        printf("Game Exit.\n");
        return 0;
    }
    if (user < 1 || user > 3) {
        printf("Incorrect user input (1~4).\n");
        return 0;
    }
}
```

```
// generate random number (0, 1, 2 -> 1, 2, 3)
computer = rand() % 3 + 1;

// 선택 출력
printf("\nYou: %s\n", choices[user]);
printf("Computer: %s\n", choices[computer]);

// 승패 판정
printf("\n[결과] ");
if (user == computer) {
    printf("same Result.\n");
} else if ((user == 1 && computer == 3) ||
           (user == 2 && computer == 1) ||
           (user == 3 && computer == 2)) {
    printf("You Win!\n");
} else {
    printf("Ooops you !\n");
}

return 0;
}
```

2. sorting

Sorting

https://github.com/kkh3363/C_Lang

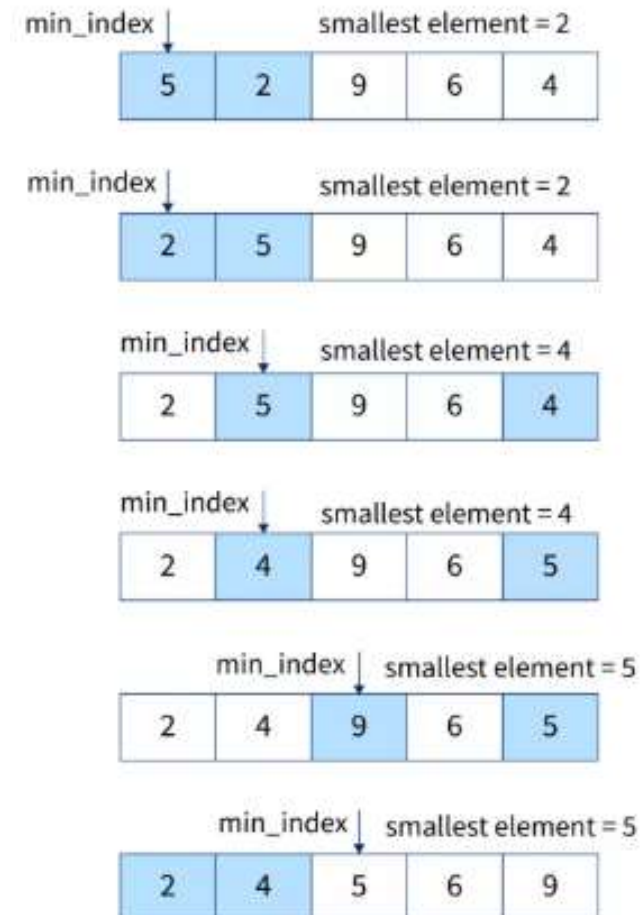
- A sorting algorithm
 - is an instruction set that rearranges elements in an array or list into a specific order, typically numerical or lexicographical. Efficient sorting is a foundational concept in computer science

Algorithm Name 	Best-Case Time	Average-Case Time	Worst-Case Time	Space Complexity	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

Sorting

https://github.com/kkh3363/C_Lang

- The selection sort algorithm in C works by repeatedly finding the minimum element from the unsorted portion of an array and swapping it with the first element of that unsorted portion



```
#include <stdio.h>

// Function to swap the values of two variables using pointers
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to perform selection sort on an array
void selectionSort(int arr[], int size) {
    int i, j;
    // Loop through the entire array except the last element
    for ( i = 0; i < size - 1; i++) {
        // Assume the current position 'i' holds the minimum value
        int min_idx = i;

        // Loop through the remaining unsorted elements to find the actual minimum
        for ( j = i + 1; j < size; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j; // Update index if a smaller element is found
            }
        }

        // Swap the found minimum element with the element at the current position 'i'
        if (min_idx != i) {
            swap(&arr[i], &arr[min_idx]);
        }
    }
}
```

```
// Function to print the elements of the array
void printArray(int arr[], int size) {
    int i;
    for ( i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int data[] = {64, 25, 12, 22, 11};
    int size = sizeof(data) / sizeof(data[0]);

    printf("Original array:\n");
    printArray(data, size);

    // Sort the array using selection sort
    selectionSort(data, size);

    printf("\nSorted array in ascending order:\n");
    printArray(data, size);

    return 0;
}
```

Thank You !